

POWERFIT

Evaluación Sumativa N°2

Implementación del modelo de clases en Python

Equipo: PowerFit Devs

Negocio asignado: PowerFit (gimnasio)

Integrantes: Javier Ojeda · Javier Concha

Repositorio: <https://github.com/javijavierojeda-hash/gym-powerfit>

Programación Orientada a Objeto Seguro · TI3V21 · Sección 114-2B-F1

Docente: Michael Arjel · INACAP Puente Alto

6 de octubre de 2026

1. Diagrama de clases y código

El diagrama de la Evaluación Sumativa N°1 se tradujo a Python con una clase por archivo en `model/`. Al programarlo encontramos detalles que había que corregir; la versión nueva está en el repositorio en `docs/diagrama/diagrama_clases.png` (editable en `DIAGRAMA_UML_POWERFIT_v2.drawio`).

PowerFit - Diagrama de clases UML (v2, Evaluación Sumativa N°2)

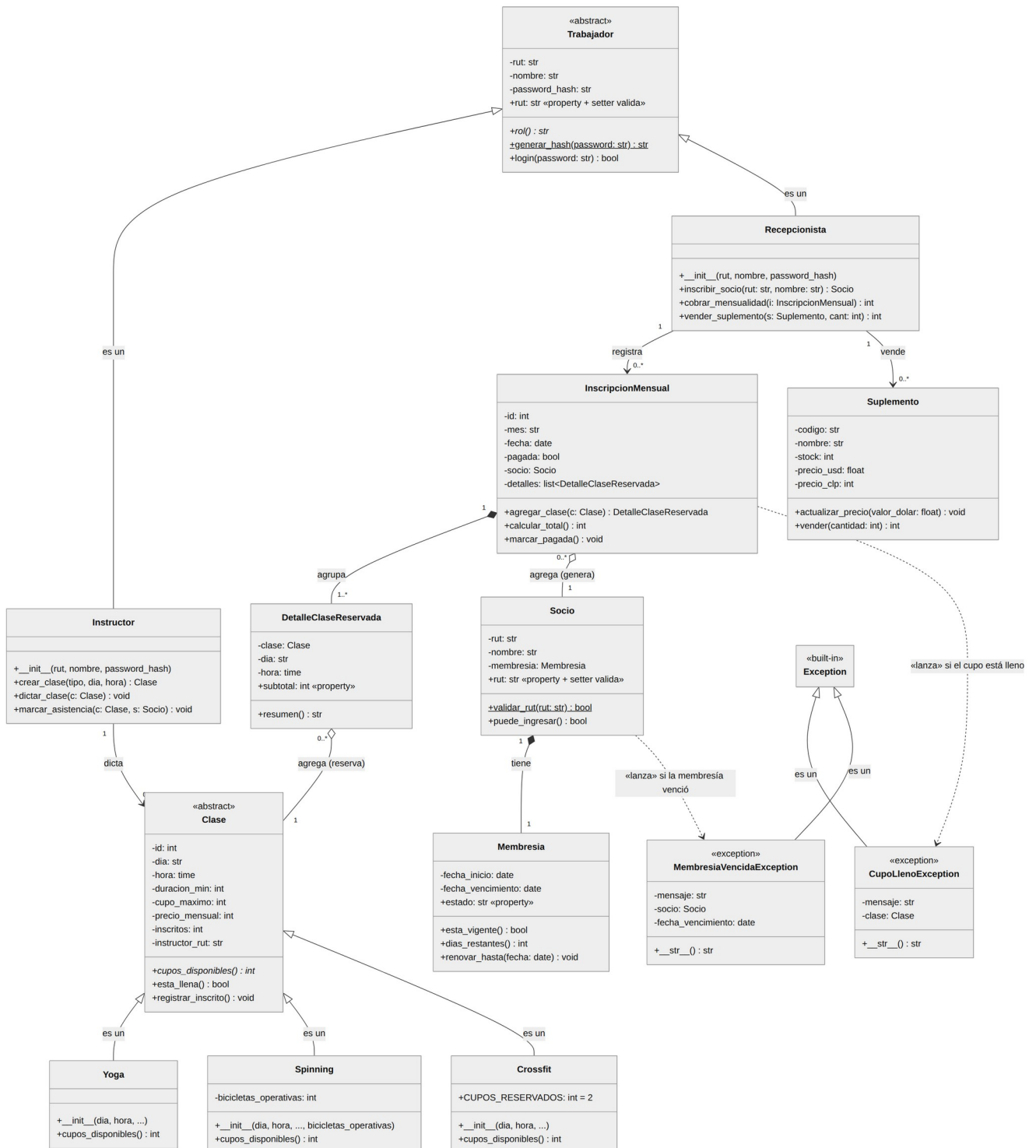


Figura 1. Diagrama de clases UML v2 de PowerFit (docs/diagrama/diagrama clases.png).

1.1 Cambios respecto del diagrama de la Evaluación Sumativa N°1

Cambio	Por qué
Agregación: IncripcionMensual ◇ Socio y DetalleClaseReservada ◇ Clase (antes asociaciones simples)	La inscripción y el detalle reciben un objeto que ya existe y que sigue existiendo sin ellos. Es una relación todo-parte débil, no una simple asociación.
<code>agregar_clase(c: Clase)</code> en vez de <code>agregar_clase(d: Detalle...)</code>	Para que la composición sea real, la inscripción (el todo) debe crear el detalle (la parte). Si el detalle se crea afuera, podría existir sin su inscripción.
<code>rut</code> como property con setter que valida (Socio y Trabajador)	La validación debe vivir en el setter: así se valida al crear la ficha y también si el RUT cambia después.
Clase agrega <code>precio_mensual</code> e <code>instructor_rut</code> ; <code>horario</code> pasa a día + hora	<code>calcular_total()</code> necesitaba un precio, y la relación "Instructor dicta Clase" necesitaba saber quién dicta. Las clases se repiten cada semana ("Yoga los lunes").
Spinning agrega <code>bicicletas_operativas</code> ; Crossfit, CUPOS_RESERVADOS	Son los datos propios que usa cada subtipo para su regla de <code>cupos_disponibles()</code> .
Instructor agrega <code>crear_clase()</code> ; IncripcionMensual agrega <code>pagada</code> y <code>marcar_pagada()</code>	Solo el instructor crea clases (la recepcionista no puede) y la inscripción no debe cobrarse dos veces.
Montos en pesos como <code>int</code> (antes <code>float</code>)	El peso chileno no tiene decimales; <code>float</code> puede producir errores de redondeo.

1.2 Herencia: Clase → Yoga, Spinning y Crossfit (también Trabajador → Instructor, Recepcionista)

La clase base es abstracta (`ABC`) y declara `cupos_disponibles()` como método abstracto. Cada subtipo hereda, llama a `super().__init__()` y sobrescribe el método con su propia regla. Se programó así porque los tres tipos comparten horario, duración y cupo, y solo cambia el cálculo de cupos (requerimiento 1).

model/clase.py (líneas 18 y 109-114) · model/yoga.py (líneas 12-33)

```
class Clase(ABC):
    @abstractmethod
    def cupos_disponibles(self) -> int: # cada subtipo DEBE implementarlo

class Yoga(Clase):
    CUPO_MAXIMO = 20
    def __init__(self, dia, hora, instructor_rut=None, id=None, inscritos=0):
        super().__init__(dia, hora, instructor_rut, id, inscritos) # constructor de Clase
    def cupos_disponibles(self) -> int: # sobrescribe el método abstracto
        return max(self.CUPO_MAXIMO - self.inscritos, 0)
```

model/spinning.py (línea 47) · model/crossfit.py (línea 34)

```
class Spinning(Clase): # regla: solo cuentan las bicicletas que funcionan
    def cupos_disponibles(self) -> int:
        return max(self.__bicicletas_operativas - self.inscritos, 0)
class Crossfit(Clase): # regla: 2 cupos reservados para clases de prueba
    def cupos_disponibles(self) -> int:
        return max(self.CUPO_MAXIMO - self.CUPOS_RESERVADOS - self.inscritos, 0)
```

1.3 Agregación: IncripcionMensual ◇ Socio y DetalleClaseReservada ◇ Clase

La agregación **recibe un objeto que ya existe**: la inscripción no crea al socio, lo recibe en su constructor; el detalle tampoco crea la clase. Se programó así porque el socio y la clase existen por sí solos: un socio puede tener varias inscripciones a lo largo de los meses y muchas reservas apuntan a la misma clase.

model/inscripcion_mensual.py (línea 38) · model/detalle_clase_reservada.py (línea 23)

```

class IncripcionMensual:
    def __init__(self, socio: Socio, mes: str, ...):
        self.__socio = socio          # AGREGACIÓN: recibe un socio que YA EXISTE

class DetalleClaseReservada:
    def __init__(self, clase: Clase, dia=None, hora=None):
        self.__clase = clase          # AGREGACIÓN: la clase sigue existiendo sin el detalle

```

1.4 Composición: Socio ♦ Membresia e IncripcionMensual ♦ DetalleClaseReservada

La composición **crea la parte dentro del todo**: el socio crea su propia membresía en el constructor, y la inscripción crea cada línea de detalle dentro de `agregar_clase()`. Así la parte no puede existir sin su todo. La lista de detalles es privada y se expone como tupla, para que nadie la modifique desde afuera (la transacción contiene sus líneas de detalle).

model/socio.py (línea 34) · model/inscripcion_mensual.py (líneas 42 y 78-98)

```

self.__membresia = Membresia(inicio, vencimiento)  # el socio CREA su membresía

self.__detalles: list[DetalleClaseReservada] = []  # lista privada de líneas de detalle
def agregar_clase(self, clase: Clase, dia=None, hora=None) -> DetalleClaseReservada:
    if clase.esta_llena():
        raise CupoLlenoException(clase)            # regla 1 del negocio
    detalle = DetalleClaseReservada(clase, dia, hora) # la inscripción CREA el detalle
    clase.registrar_inscrito()
    self.__detalles.append(detalle)
    return detalle

```

2. Evidencia de funcionamiento (main.py)

Capturas de la ejecución de `python main.py` en la raíz del repositorio. El script solo usa la librería estándar (Python 3.7 o superior) y corre de principio a fin sin errores no controlados.

2.1 Los tres subtipos con su método distinto

Clase y método: `Clase.cupos_disponibles()`, sobrescrito en `Yoga`, `Spinning` y `Crossfit`. `main.py` recorre la lista con un solo `for` y llama al método sin preguntar el tipo de cada objeto; cada subtipo responde con su regla (polimorfismo).

```
C:\gym-powerfit> python main.py
...
=====
REQUISITO 1 - Tres subtipos con su método distinto (polimorfismo)
=====
Método abstracto: Clase.cupos_disponibles() -> sobrescrito en model/yoga.py,
model/spinning.py y model/crossfit.py. Cada subtipo llama a super().__init__().

Yoga      cupo máximo 20 | 60 min | cupos_disponibles() = 20
Spinning  cupo máximo 15 | 45 min | cupos_disponibles() = 13
Crossfit   cupo máximo 12 | 50 min | cupos_disponibles() = 10

Regla de cada subtipo:
- Yoga:      cupo máximo - inscritos
- Spinning:  bicicletas operativas (13) - inscritos
- Crossfit:  cupo máximo - 2 cupos reservados - inscritos
```

Figura 2. Requisito 1: polimorfismo de `cupos_disponibles()`.

main.py

```
for clase in [yoga, spinning, crossfit]:      # sin if sobre el tipo
    disponibles = clase.cupos_disponibles()    # cada subtipo usa su propia regla
```

2.2 El dato con validación en el setter

Clase y método: `Socio.rut` (`@rut.setter` en `model/socio.py`, línea 48). El constructor asigna `self.rut = rut`, así que la validación módulo 11 ocurre siempre en el setter: al crear la ficha y al intentar cambiar el RUT.

```
=====
REQUISITO 2 - Dato con validación en el setter: Socio.rut
=====
Setter: @rut.setter en model/socio.py (valida con el algoritmo módulo 11).

RUT válido   '12.345.678-5' -> ficha creada: Ana Perez (12.345.678-5)
RUT inválido '12.345.678-9' -> ficha NO creada (ValueError: RUT invalido: 12.345.678-9)
Cambio a 'abc' rechazado (ValueError: RUT invalido: abc); RUT se mantiene: 12.345.678-5
```

Figura 3. Requisito 2: RUT válido aceptado; RUT inválido y cambio inválido rechazados por el setter.

model/socio.py (líneas 48-56)

```
@rut.setter
def rut(self, valor: str) -> None:
    if not Socio.validar_rut(valor):          # algoritmo módulo 11
        raise ValueError(f"RUT invalido: {valor}") # el atributo no cambia
    self.__rut = limpiar_rut(valor)           # guarda el RUT normalizado
```

2.3 La transacción con sus líneas de detalle

Clase y método: `InscripcionMensual.agregar_clase()` crea cada `DetalleClaseReservada`; `calcular_total()` suma los subtotales y `Recepcionista.cobrar_mensualidad()` cobra y renueva la membresía.

```
=====
REQUISITO 3 - Transacción con sus líneas de detalle: InscripcionMensual
=====
InscripcionMensual.agregar_clase(clase) CREA cada DetalleClaseReservada dentro
de la inscripción (composición) y recibe la Clase que ya existe (agregación).

Inscripcion 2026-10 de Ana Perez (2 clases)
Línea 1: Yoga · lunes 09:00          $15.000
Línea 2: Crossfit · miercoles 20:00  $22.000
TOTAL calcular_total()              $37.000
Recepcionista.cobrar_mensualidad() -> cobrado $37.000; Membresia vigente hasta 31-10-2026
```

Figura 4. Requisito 3: inscripción mensual con dos líneas de detalle y su total.

2.4 Las dos reglas del negocio (excepciones propias)

Regla 1: `CupoLlenoException`, lanzada por `InscripcionMensual.agregar_clase()` (`model/inscripcion_mensual.py`, línea 93). **Regla 2:** `MembresiaVencidaException`, lanzada por `Socio.puede_ingresar()` (`model/socio.py`, línea 82). Ambas heredan de `Exception`, se provocan a propósito y `main.py` las captura con `try/except` específicos (no con un `except` genérico), informa el error y sigue ejecutándose.

```
=====
REQUISITO 4 - Las dos reglas del negocio (excepciones propias)
=====
Regla 1: no inscribir en una clase con el cupo lleno.
Lanzada por: InscripcionMensual.agregar_clase() (model/inscripcion_mensual.py)
-> CupoLlenoException capturada: La clase ya alcanzo su cupo maximo: Crossfit miercoles 20:00
La inscripción quedó sin cambios: 0 clases reservadas.

Regla 2: no dejar entrar a un socio con la membresía vencida.
Lanzada por: Socio.puede_ingresar() (model/socio.py)
-> Ana Perez: puede ingresar (Membresia vigente hasta 31-10-2026)
-> MembresiaVencidaException capturada: La membresia del socio esta vencida: Agustin Reyes (vencio el 05-09-2026)
```

Figura 5. Requisito 4: ambas excepciones capturadas; el programa continúa.

```
main.py
try:
    otra_inscripcion.agregar_clase(crossfit)          # clase llena a propósito
except CupoLlenoException as error:
    print(f"-> CupoLlenoException capturada: {error}")
try:
    socio.puede_ingresar()                             # socio con membresía vencida
except MembresiaVencidaException as error:
    print(f"-> MembresiaVencidaException capturada: {error}")
```

3. Uso de inteligencia artificial

Usamos **Claude (Anthropic)**, a través de **Claude Code**, como asistente de programación. El equipo aportó el negocio, los requerimientos, el diagrama UML de la Evaluación Sumativa N°1 y el repositorio del profesor como referencia de arquitectura, y tomó las decisiones de diseño.

3.1 Qué partes propuso la IA

- La primera versión de las clases de `model/` a partir de nuestro diagrama: atributos privados, properties, el algoritmo módulo 11 del RUT y las dos excepciones.
- El script `main.py`, las pruebas automáticas (`tests/`, 73 pruebas) y la documentación del repositorio.
- Fuera del alcance de esta evaluación: la capa de datos (`dao/`) y un prototipo web que se usará en la Evaluación Sumativa N°3.

3.2 Cómo validamos lo que entregó

No aceptamos el código sin revisarlo: (1) lo comparamos clase por clase contra el diagrama y contra esta pauta; (2) ejecutamos las pruebas automáticas después de cada cambio; (3) clonamos el repositorio desde cero y ejecutamos `python main.py`, igual que lo hará el docente; y (4) revisamos la versión mínima de Python con la herramienta `vermin`.

3.3 Ejemplos concretos: adoptado, modificado y descartado

Propuesta de la IA	Decisión	Razón técnica
Validar el RUT en el constructor de Socio (if + raise antes de asignar).	Modificado	La pauta y el principio de encapsulamiento exigen que la validación viva en el setter. En el constructor solo protegía la creación; si el RUT se cambiaba después, quedaba sin validar. Se movió a <code>@rut.setter</code> y el constructor usa <code>self.rut = rut</code> .
agregar_clase(detalle) recibía un DetalleClaseReservada creado afuera.	Modificado	Eso es agregación, no composición: el detalle podía existir sin su inscripción. Ahora <code>agregar_clase(clase)</code> crea el detalle dentro del todo y el diagrama se actualizó.
Que Yoga y Crossfit heredaran el constructor de Clase de forma implícita (sin <code>__init__</code> propio).	Modificado	Funciona en Python, pero no muestra la llamada a <code>super().__init__()</code> que pide la pauta. Se agregó un constructor explícito en los cinco subtipos.
Usar anotaciones de tipo modernas (<code>str None</code> , <code>list[str]</code>).	Modificado	Al revisar, vimos que fallan en Python 3.7 a 3.9 y no sabemos qué versión tiene el computador del docente. Se agregó <code>from __future__ import annotations</code> y se verificó que la versión mínima es 3.7.
Calcular el estado de la membresía como property, en vez de guardarlo como texto ("vigente"/"vencida") como estaba en nuestro UML.	Adoptado	Un texto guardado queda desactualizado apenas pasa la fecha de vencimiento. Calculado a partir de las fechas, siempre es correcto.
Agregar una capa web y base de datos en esta entrega.	Descartado para la ES2	La pauta evalúa solo el modelo de clases. <code>main.py</code> se dejó independiente de la base de datos y de internet para que corra en cualquier computador; lo demás queda para la Evaluación Sumativa N°3.
Guardar las contraseñas con un hash PBKDF2 con "salt" y compararlas con <code>hmac.compare_digest</code> .	Adoptado	Es más seguro que comparar hashes con <code>==</code> : dos contraseñas iguales generan hashes distintos y la comparación tarda lo mismo aunque falle (evita ataques por tiempo de respuesta).

Conclusión: la IA aceleró la escritura del código, pero varias de sus propuestas iniciales no cumplían la pauta o el diagrama. Detectarlas exigió entender herencia, composición, agregación y encapsulamiento, y validar el resultado ejecutándolo.